

Linguaggi di programmazione

Paradigma imperativo (procedurale):

Le caratteristiche fondamentali dei linguaggi imperativi sono legate all'architettura di Von Neumann, questa ha due componenti:

- **Memoria:** è la componente passiva che contiene tutti i dati.
- **Processore:** è la componente attiva che esegue tutte le computazioni e legge e scrive nelle celle di memoria, ciò implica che le **variabili** sono da immaginarsi come astrazioni della memoria.

Adottano uno **stile prescrittivo**, il quale prescrive le operazioni che il processore deve eseguire per modificare lo stato del sistema. Le istruzioni vengono eseguite nell'ordine in cui appaiono (escluse le strutture di controllo). Vengono utilizzati sia attraverso compilazione che interpretazione.

Struttura:

- **Dichiarazione:** contente le variabili, i tipi e le procedure.
- **Definizione:** è la parte in cui gli algoritmi utilizzati vengono implementati usando le istruzioni del linguaggio.

Istruzioni:

Le istruzioni possono essere:

- Istruzioni di I/O.
- Istruzioni di assegnamento (in questo caso le variabili sono astrazioni delle celle di memoria).
- Istruzioni per il flusso di controllo.

Usi:

- Gestire applicazioni con alto livello di astrazione.
- Sviluppare programmi più semplici da scrivere e capire.

Paradigma logico funzionale:

Sono linguaggi ad alto livello che sono basati su concetti matematici e non su astrazioni della macchina di Von Neumann, sono generati per manipolazione simbolica e non numerica e adottano uno stile dichiarativo. Nel paradigma logico funzionale non c'è una netta distinzione tra programmi e strutture dati.

Paradigma logico:

Il paradigma logico si basa sul concetto di deduzione logica, costruito a partire dalla logica formale per creare dei "ragionamenti" logici. Definire linguaggio logico vuol dire definire come un programmatore può esprimere una parte della conoscenza e che tipo di controllo può essere usato nel processo di deduzione.

La scrittura di un programma consiste nel:

- Descrivere un problema come un insieme di espressioni logiche del linguaggio.
- Interrogare (query) il sistema che farà deduzioni sulla base delle informazioni che gli forniamo.

Si utilizza uno stile dichiarativo attraverso il quale è possibile esprimere la conoscenza indipendentemente dal suo uso (WHAT and not HOW).

Paradigma funzionale:

I paradigmi funzionali sono basati sul concetto matematico di funzione. Una funzione è un'associazione tra due insiemi, associa un elemento del primo insieme (dominio) ad un elemento del secondo insieme (codominio). Dopo aver definito una funzione questa viene applicata ad un argomento del dominio in modo da ritornare il range di elementi del codominio (evaluation).

Nei linguaggi funzionali (puri) le uniche operazioni utilizzate sono le funzioni, il valore restituito da una funzione dipende solamente dagli argomenti e dalla funzione stessa e non dallo stato del sistema, le variabili sono viste come costanti matematiche e sono quindi immutabili. L'essenza del paradigma funzionale è di combinare funzioni componendole tra di loro affiancato all'uso del concetto di ricorsione.

Regole di inferenza:

Un insieme di regole di inferenza costituisce la base di un **calcolo logico**. Lo scopo di un calcolo logico è di manipolare delle formule logiche in modo completamente sintattico al fine di stabilire una connessione tra un insieme di formule di partenza e un insieme di conclusioni.

Logica proposizionale:

La logica proposizionale si occupa delle conclusioni che possiamo trarre da un insieme di proposizioni, a cui è legata una **funzione di verità** (valutazione) V che associa un valore di verità a ogni elemento di P . La funzione di valutazione è il ponte tra la sintassi e la semantica di un linguaggio logico.

Le proposizioni possono essere combinate tra loro attraverso dei connettivi logici ($\wedge, \vee, \neg, \rightarrow, \leftrightarrow$). Chiamiamo FBF l'insieme di tutte le formule formate da elementi di P e le loro combinazioni. Le formule atomiche in P e le loro negazioni vengono anche chiamati letterali (positivi e negativi).

Calcolo logico:

Un calcolo logico ci dice come generare nuove formule (cioè espressioni sintattiche) a partire da un insieme di partenza (gli assiomi) A .

Se:

- $A \subset WFF$
- $b \subset WFF$
- $A = S$

Il calcolo logico deve quindi garantire che le nuove formule generate siano “vere” solo se l'insieme di assiomi consiste solo di formule “vere” (processo di dimostrazione).

Calcolo proposizionale:

Il calcolo proposizionale è basato su una serie di regole di inferenza che ci permettono di ottenere delle nuove formule a partire da un insieme di assiomi.

Una regola di inferenza ha la seguente formula generale:

$$\frac{F_1, F_2, \dots, F_K}{R} [\text{nome regola}]$$

dove

- F_i rappresenta una regola (vera) in FBF.
- R è la formula generata da inserire in FBF.
- [nome regola] ci dice che regola di inferenza stiamo usando.

Modus Ponens:

La regola del modus ponens ci permette di aggiungere le conclusioni di un'implicazione al nostro insieme di formule ben formate “vere” da FBF.

$$\frac{p \rightarrow q, p}{q} [\text{modus ponens}]$$

Esempio:

- $p \rightarrow q$: se piove allora la strada è bagnata.
- p : piove.
- q : allora la strada è bagnata.

Modus Tollens:

La regola sintattica del modus tollens ci permette di aggiungere la premessa negata di una «regola» al nostro insieme di formule ben formate “vere”.

$$\frac{p \rightarrow q, \neg q}{\neg p} [\text{modus tollens}]$$

Esempio:

- $p \rightarrow q$: se piove allora la strada è bagnata.
- $\neg p$: la strada non è bagnata.
- $\neg q$: allora non piove.

Introduzione ed eliminazione dell' $\wedge$

La regola sintattica dell'eliminazione della congiunzione ci permette di aggiungere all'insieme FBF i singoli componenti di una congiunzione.

$$\frac{p_1 \wedge p_2 \wedge \dots \wedge p_n}{p_i} \text{ [rimozione di } \wedge \text{]}$$

Esempio:

- Piove e la strada è bagnata.
- (segue che) piove.

Se è vera una congiunzione, allora sono vere anche le singole proposizioni congiunte. Ad esempio, se la congiunzione $A \wedge B$ è vera, allora anche la proposizione A è vera.

$$\frac{p_1, p_2, \dots, p_n}{p_{i p_1} \wedge p_2 \wedge \dots \wedge p_n} \text{ [introduzione di } \wedge \text{]}$$

Introduzione dell'or:

La regola sintattica dell'introduzione della disgiunzione ci permette di aggiungere i singoli componenti di una formula complessa.

$$\frac{p}{p \vee q} \text{ [introduzione di } \vee \text{]}$$

Esempio:

- Piove.
- Piove o c'è vita su Marte.

Varie e tautologie:

- $\frac{p \vee \neg p}{\text{vero}}$ [terzo escluso]
 - se p è vera, allora p è falsa;
 - se p è falsa, allora p è vera.
- $\frac{p \wedge \text{vero}}{p}$ [eliminazione di $\wedge$]
- $\frac{\neg \neg p}{p}$ [eliminazione di $\neg$]
- $\frac{p \wedge \neg p}{q}$ [contraddizione] (da una contraddizione si può trarre qualsiasi conseguenza)

Principio di risoluzione:

È una regola di inferenza generalizzata che fa uso del modus ponens usata in congiunzione con quella del terzo escluso per dimostrare qualcosa per assurdo.

Se si arriva a generare la clausola vuota ($\perp$) corrisponde a dimostrare che il mio insieme di FBF contiene una contraddizione:

$$\frac{\neg r, r}{\perp}$$

Sintassi e semantica:

Un calcolo logico fornisce una manipolazione sintattica (simbolica)

- L'operatore di derivazione $\vdash$ è un operatore sintattico

La semantica di un insieme di formule dipende dalla funzione di valutazione V .

- Simmetricamente viene introdotto l'operatore di "conseguenza logica" denotato da $\models$.

Teorema di completezza e validità:

Dato un calcolo logico:

$$S \vdash f \text{ se e solo se } S \models f$$

Dove S è un insieme di formule iniziale ed f è una fbf; il tutto in dipendenza da una particolare funzione di verità V . Una V che rende vere tutte le formule di S viene detta **modello di S** .

Logica del primo ordine:

La logica del primo ordine introduce le nozioni di:

- Variabile.
- Costante.
- Relazione (o predicato).
- Funzione.
- Quantificatore.

Un linguaggio del primo ordine è quindi composto da termini costruiti a partire da:

- V insieme di simboli di variabili.
- C insieme di simboli di costante.
- R insieme di simboli di relazione o predicati di varia arità.
- F insieme di simboli di funzione di varia arità.
- Connettivi logici e simboli di quantificazione $\forall$ (universale) e $\exists$ (esistenziale).

Sintassi della logica del primo ordine:

La sintassi della logica del primo ordine è ricorsiva:

- I termini più semplici sono i **predicati**, ovvero relazioni cartesiane su C del tipo:

$$r \subseteq C_0 \times C_1 \times \dots \times C_n$$

Si scrivono come $r(c_1, c_2, \dots, c_n)$.

- Le **funzioni** sono definite col seguente dominio e codominio:

$$f : C_0 \times C_1 \times \dots \times C_n \rightarrow C$$

Si scrivono come $f(c_1, c_2, \dots, c_n)$.

NB:

Una funzione $f(c_1, c_2, \dots, c_n)$ non è una FBF

- Le **FBF** di un FOL si costruiscono ricorsivamente nel seguente modo:
 - Un **termine** t_j può essere un elemento di C , di V o l'applicazione di una funzione f .
 - Un termine costituito da un predicato $r(t_1, t_2, \dots, t_n)$ dove ogni t_i è un termine è una FBF.
 - Elementi di FBF connessi da connettivi logici appartengono a FBF e sono espressi come $t(t_1, t_2, \dots, t_n)$.
 - Le formule $\forall x. t(t_1, t_2, \dots, t_n)$ e $\exists x. t(t_1, t_2, \dots, t_n)$ appartengono a FBF.

Regole del calcolo logico:

- Eliminazione del quantificatore universale

$$\frac{\forall x. T(\dots, x, \dots), c \in C}{T(\dots, c, \dots)}$$

- Introduzione del quantificatore universale

$$\frac{T(\dots, c, \dots), c \in C}{\exists x. T(\dots, x, \dots)}$$

- $\exists x. \neg T(\dots, x, \dots) \equiv \neg \forall x. T(\dots, x, \dots)$
- $\forall x. \neg T(\dots, x, \dots) \equiv \neg \exists x. T(\dots, x, \dots)$

Prolog

Prolog è basato su una restrizione della logica del primo ordine, ha uno stile dichiarativo. Si usa per determinare per quali valori un'affermazione è vera e, nel caso che lo sia, quali vincoli sui valori attribuiti alle variabili hanno generato la risposta.

FBF e forma normale a clausole

Ogni formula ben formata di un linguaggio logico del primo ordine può essere riscritta in forma normale a clausole.

Vi sono due forme normali a clausole:

1. **Forma normale congiunta:** la formula è una congiunzione di disgiunzioni di predicati o di negazioni di predicati

$$\bigwedge_i \left(\bigvee_j L_{ij} \right)$$

2. **Forma normale disgiunta:** La formula è una disgiunzione di congiunzioni di predicati o di negazioni di predicati.

$$\bigvee_j \left(\bigwedge_i L_{ij} \right)$$

Dove: $L_{ij} \equiv P_{ij}(x, y, \dots, z)$ o $L_{ij} \equiv \neg Q_{ij}(x, y, \dots, z)$

Clausole di Horn:

Le clausole che hanno al più un solo letterale positivo sono dette **clausole di Horn**:

- Non tutte le FBF possono essere trasformate in un insieme di clausole di Horn.
- I programmi Prolog sono collezioni di clausole di Horn.

In particolare, possiamo classificare le clausole di Horn come:

- **Fatti:** $A \leftarrow$
- **Regole:** $A \leftarrow B_1 \wedge B_2 \wedge \dots \wedge B_n$
- **Goals:** $\leftarrow B_1 \wedge B_2 \wedge \dots \wedge B_n$
- **Contraddizione:** $\leftarrow$

In Prolog:

- **Fatti:** $A.$
- **Regole:** $A :- B_1, B_2, \dots, B_n$
- **Goals:** $:- B_1, B_2, \dots, B_n$
- **Contraddizione:** `fail`

Sintassi di Prolog:

Prolog è un linguaggio dichiarativo contenente per lo più fatti e regole e poche istruzioni.

- **Fatti:** Asserzioni vere nel contesto che stiamo descrivendo.
- **Regole:** Ci danno gli strumenti per dedurre nuovi fatti da quelli esistenti.

Un programma Prolog ci dà informazioni su un sistema ed è normalmente chiamato base di conoscenza, non viene eseguito bensì viene interrogato e risponde in modo affermativo o negativo (SI/NO)

Sintassi:

Un programma Prolog è costituito da un insieme di clausole della forma:

a .	% FATTO o ASSERTIONE
c :- b₁, b₂, ..., b_n .	% REGOLA
:- q₁, q₂, ..., q_m .	% GOAL
?- q₁, q₂, ..., q_m .	% QUERY

In cui a, b_i, c, q_i sono **termini** (composti).

Termini:

Le espressioni di Prolog sono chiamate termini e si dividono in:

- **Operatori**
 - % o /* ... */ (commenti) := si usa per confrontare ciò che si trova a destra e a sinistra dell'ugale
 - , (and)
 - ; (or)
 - = (unificazione)
 - ?- (query)
 - | (delimitatore per coda delle liste)
 - ! (cut)
 - is ("assegnamento")
- **Atomi**
 - Sequenza di caratteri alfanumerici che inizia con un carattere minuscolo (può contenere "_").
 - Qualsiasi cosa racchiusa tra apici.
 - Un numero.
 - Una stringa (in SWI Prolog).
 - ...
- **Variabili logiche:**
 - Una variabile (logica) è una sequenza alfanumerica che inizia con un carattere maiuscolo o con il carattere.
 - Le variabili composte solo da "_" sono chiamate anonime.
 - Vengono istanziate con il procedere del programma (dimostrazione del teorema).
- **Termini Composti:**
 - Un funtore (simbolo di funzione o di predicato definito come atomo).
 - Una sequenza di termini racchiusi tra parentesi tonde e separati da virgole. Questi sono chiamati argomenti del funtore. (non ci deve mai essere uno spazio tra il funtore e la parentesi di sinistra; questo per via di caratteristiche molto sofisticate del sistema di parsing di Prolog).

Fatti o predicati:

Un fatto (predicato) consiste in:

- Un nome di predicato che deve iniziare con la lettera maiuscola.
- Zero o più argomenti.
- I fatti devono essere terminati con un punto.

ES.

```
product(n1, n2) .  
    animal(dog) .  
parent(p1, p2) .
```

Regole:

In Prolog una regola consiste di una testa (**head**) e di un corpo (**body**).

- Testa e corpo sono collegati dall'operatore :-
- La testa di una regola corrisponde al conseguente di un'implicazione logica.
- Il corpo di una regola corrisponde all'antecedente di un'implicazione logica.
- Le regole Prolog corrispondono alle clausole di Horn, ovvero hanno un solo termine (predicato).

Esempio:

“il pesce è un animale che ha le squame” →

```
pesce(X) :- animale(X), squamato(X) .
```

In Prolog una relazione può essere definita da più regole (clausole) che hanno lo stesso predicato come conclusione.

Esempio:

```
parent(X, Y) :- father(X, Y) .  
parent(X, Y) :- mother(X, Y) .
```

Le regole (e i fatti) sono implicitamente connesse dall'and.

Ricorsione:

Una relazione può anche essere definita ricorsivamente. In questo caso la definizione richiede almeno due proposizioni: una è quella ricorsiva che corrisponde al caso generale, l'altra esprime il caso particolare più semplice.

Esempio:

```
ancestor(X, Y) :- parent(X, Y) .  
ancestor(X, Y) :- parent(Z, Y), parent(X, Z) .
```

Unificazione:

L'operazione di istanziamento di variabili durante la “prova” di un predicato è il risultato di una procedura particolare, detta unificazione. Dati due termini, la procedura di unificazione crea un insieme di sostituzioni delle variabili, Questo insieme di sostituzioni permette di rendere uguali i due termini.

Tradizionalmente la procedura di unificazione costruisce un insieme di sostituzioni chiamato **most general unifier** ed indicato con Mgu. Una sostituzione è indicata come una sequenza (ordinata) di coppie variabile/valore.

<i>Mgu</i> (42, 42)	⇒ {}
<i>Mgu</i> (42, X)	⇒ {X/42}
<i>Mgu</i> (X, 42)	⇒ {X/42}
<i>Mgu</i> (foo(bar, 42), foo(bar, X))	⇒ {X/42}
<i>Mgu</i> (foo(Y, 42), foo(bar, X))	⇒ {Y/bar, X/42}
<i>Mgu</i> (foo(bar(42), baz), foo(X, Y))	⇒ {X/bar(42), Y/baz}
<i>Mgu</i> (foo(X), foo(bar(Y)))	⇒ {X/bar(Y), Y/_G001}

Il MGU corrisponde quindi al risultato finale della procedura di valutazione di Prolog. Per richiamare l'unificazione si usa l'operatore =.

ES:

```
?- foo(bar, 42 = foo(bar, X).
X = 42
True
```

Liste:

Le liste vengono definite in Prolog con un insieme di elementi separati da virgole e racchiuse tra []. La lista vuota si rappresenta con [].

Ogni lista non vuota può essere divisa in due parti: una testa ed una coda.

- La testa è il primo elemento della lista
- La coda rappresenta la restante parte ed è sempre una lista.

ES:

[a, b, c]	a è la testa e [b, c] la coda
[a]	a è la testa e [] la coda
[[a, b], [c], d]	[a, b] è la testa e [[c], d] la coda

L'operatore |

L'operatore | viene usato per distinguere l'inizio e la coda di una lista.

```
?- [X | Ys] = [mia, vincent, jules, yolanda].
X = mia
Ys = [vincent, jules, yolanda]
Yes

?- [X, Y | Zs] = [the, answer, is, 42].
X = the
Y = answer
Zs = [is, 42]
Yes

?- [X, 42 | _] = [41, 42, 43, foo(bar)].
X = 41
Yes
```

Interprete Prolog:

Consult:

Il predicato `consult()` si usa per inizializzare o caricare un insieme di fatti nell'ambiente di Prolog. Prende come argomento il path di un file `.pl` o `user`, che consente l'inserimento di fatti e righe dalla shell.

Reconsult:

`reconsult()` si usa per ricaricare un file nell'ambiente di Prolog, rimuove quindi i vecchi predicati e li sostituisce con le nuove definizioni. Prende gli stessi input di `consult()`.

Esecuzione di un programma:

La computazione eseguita da Prolog corrisponde al tentativo di dimostrare che la formula segua logicamente da un programma, ovvero che la formula sia un teorema.

Inoltre, si deve determinare una sostituzione per le variabili della query perché questa segua logicamente dal programma.

Dato un insieme di clausole di Horn è possibile derivare la clausola vuota solo se ce n'è almeno una senza testa, cioè almeno una query G_0 da provare. Si deve quindi dimostrare che da $P \cup G_0$ si può derivare la clausola vuota, cioè dimostrare per assurdo una soluzione.

Risoluzione ad input lineare (SDL):

Prolog dimostra la veridicità di una query eseguendo una serie di passi di risoluzione. La risoluzione avviene sempre fra l'ultimo goal derivato in ciascun passo e una clausola di programma.

ES:

A partire dal goal

$$G_i \equiv ?- A_{i,1}, A_{i,2}, \dots, A_{i,n}$$

E dalla regola:

$$A_r \equiv ?- B_{r,1}, B_{r,2}, \dots, B_{r,n}$$

Si cerca un unificatore ($=$) tale che $\sigma[A_r] = \sigma[A_{i,1}]$ in modo da ottenere un nuovo goal G_{i+1} :

$$G_{i+1} \equiv ?- B'_{r,1}, B'_{r,2}, \dots, B'_{r,n}, A'_{i,1}, A'_{i,2}, \dots, A'_{i,m}$$

Dove A' e B' sono i risultati delle sostituzioni $\sigma[A_{i,1}] = \sigma[A'_{i,1}]$ e $\sigma[B_{r,1}] = \sigma[B'_{r,1}]$

Il risultato finale può essere

- **Successo:** se viene generata la clausola vuota, quindi per n finito $G_n \equiv :-$
- **Insuccesso finito:** se per n finito $G_n \not\equiv :-$ e non si può derivare nessun nuovo risolvente
- **Insuccesso infinito:** se è sempre possibile derivare nuovi risolventi tutti diversi da $:-$

Selezione di un sub-goal:

Essendoci più clausole di programma utilizzabili per la risoluzione del goal se ne deve selezionare una. Ci sono due strategie di ricerca:

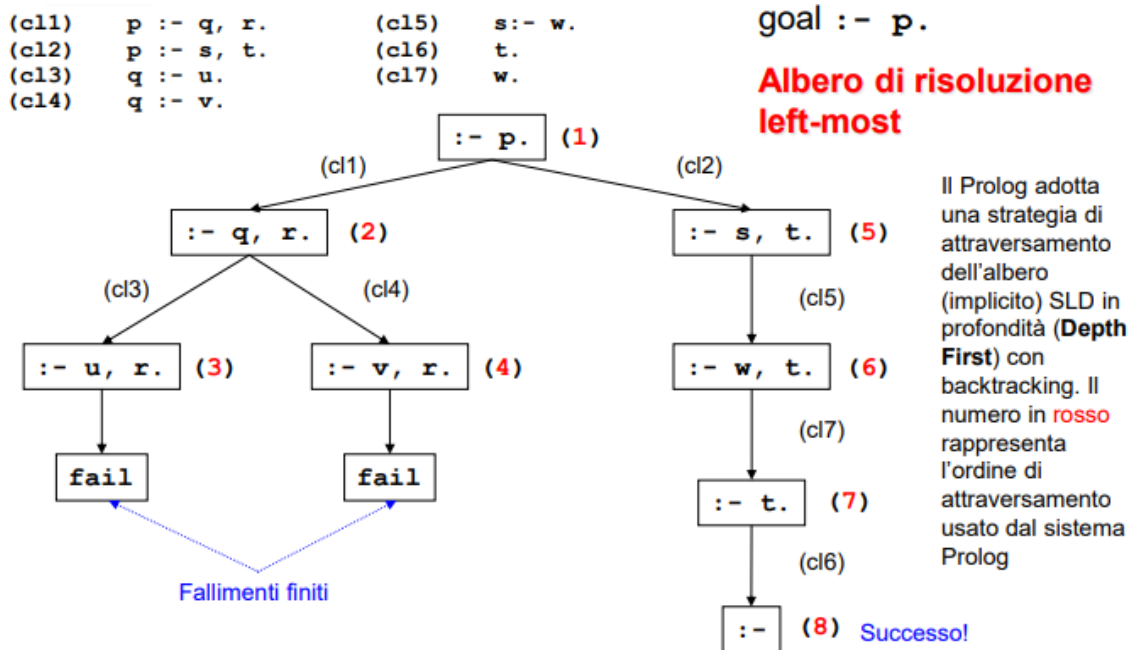
- **Depth first:** si sceglie una clausola e si mantiene fissa questa scelta, finché non si arriva alla clausola vuota o alla impossibilità di fare nuove risoluzioni; in questo ultimo caso si riconsiderano le scelte fatte precedentemente.
- **Breadth first:** si considerano in parallelo tutte le possibili alternative.

Alberi di derivazione SLD:

Dato un programma logico P , un goal G_0 e una regola di selezione R , un albero $P \cup \{G_0\}$ via R :

- La radice è il goal G_0 .
- Ogni nodo è un goal (possibilmente vuoto).
- Dato un nodo un sub-goal A_m che viene selezionato da R ha un nodo figlio per ogni clausola del tipo.

- $C_i \equiv A_i :- B_{i,1}, \dots, B_{i,n}$
- $C_k \equiv A_k$



Ad ogni ramo di un albero SLD corrisponde una derivazione SLD

- Ogni ramo che termina con il nodo vuoto :- rappresenta una derivazione SLD di successo.

La regola di calcolo influisce sulla struttura dell'albero per quanto riguarda sia l'ampiezza sia la profondità. Tuttavia, non influisce su correttezza e completezza; quindi, qualunque sia R , il numero di cammini di successo (se in numero finito) è lo stesso in tutti gli alberi SLD costruibili per $P \cup \{G_0\}$. R influenza solo il numero di cammini di fallimento.

Modello di esecuzione Prolog:

$p :- q, r.$

Scomponiamo il problema nei sotto problemi p e q . Ognuno di questi sub-goal può essere visto come una chiamata a una procedura. La regola invece viene vista come la definizione di una procedura in cui la testa è l'intestazione e la parte destra il corpo.

Controllo di esecuzione di programma:

Nel caso in cui la dimostrazione di un sub-goal fallisse allora Prolog sceglie un'alternativa nella lista delle clausole. Possiamo controllare direttamente questa operazione con l'operatore (cut !). È un'operazione molto complessa senza valore logico ma solo procedurale.

Utilizzo dell'operatore !:

Si possono distinguere due usi dell'operatore cut:

- **Green Cuts:** utili per esprimere determinismo e mantengono la semantica.
- **Red Cuts:** usati solo per efficienza omettono alcune condizioni esplicite e modificano la semantica.

